Chui Twin: Technical Appendix

Longaeva AI Hackathon, finals (October 2026). Code: https://github.com/Ben369369/Chui-Twin-Financial-Modelling

This appendix covers how the system works, how it was tested and where it falls short. It is meant for readers who want to check the claims. Full model math is in `docs/model.md`, and data sources and terms are in `provenance.md`.

1. What the system does

Chui Twin turns a public company into a runnable operating model. Public data is treated as **noisy sensors of hidden operating states**: demand, capacity, price, input cost, labor, sentiment and expansion. One shared engine:

1. estimates those states each quarter,
2. links them to the KPIs the company reports through a causal graph,
3. simulates the next quarters with Monte Carlo,
4. traces every forecast back to the data behind it,
5. scores every twin the same way against simple and expert baselines.

The claim is about **the system, not one company**. Seventeen twins in six industries (five industry packs plus cruise lines) run on the same engine. Fifteen of them were built by a factory from the ticker alone, with no per-company tuning: fourteen by rules, and one (Carnival, an industry no pack covers) with an LLM reading the 10-K (section 10).

2. Alpha, as we define it

The brief asks us to simulate the system that generates alpha, as we define it.

Definition. Alpha, here, is an information edge on a company's next reported results: knowing, before the release and from data that was public at the time, where the reported KPIs will land relative to what the market expects. Prices move on the gap between results and expectations, so the edge lies in forecasting that gap, not the level.

The system that generates it is the company's own operations. A few hidden states (demand, capacity, price, input cost, labor, sentiment, expansion) drive the KPIs the company reports. Those states move weeks before they are reported, and public data watches them move: TSA checkpoint counts and jet fuel prices for an airline; restaurant sales, wages, store counts and news tone for a restaurant chain. The twin simulates that system. It estimates the states from these sensors, carries them through a causal graph to the KPIs, simulates 10,000 paths, and reads off the distribution of the reported number and the probability that it beats the expectation. The reasoning is shown, not asserted: every forecast breaks down into drivers and the data series behind them (section 5), and live forecasts are sealed before the outcome (section 8).

From data to alpha is a chain of three links, and each needs its own evidence:

Link	Evidence so far
1. Public sensors track the hidden states	The twin beats "same as last quarter" on 25 of 37 KPI series, and "same quarter last year" on 35 of 37 (section 7)
2. The twin knows more than the expectation	Beats management guidance on accuracy: CRPS skill +0.29 (Delta revenue, 14 quarters), +0.31 (Chipotle comps, 20 quarters). Loses to analyst consensus: −0.37 (Chipotle comps, 6 quarters)
3. That knowledge is not already in the price	Not tested

The sharper test of link 2 is whether the twin calls the **direction** of the surprise (`scripts/reports/surprise_direction.py`, `docs/surprise_direction.md`). A call is the sign of (twin median − expectation), scored walk-forward against the sign of (actual − expectation):

Twin, KPI	Against	Quarters	Twin called it	Best constant rule	p (vs coin)
Delta, revenue growth	guidance	14	9 (64%)	50%	0.21
Chipotle, comps	guidance	18	13 (72%)	72% ("always beat")	0.05
Chipotle, comps	consensus	6	3 (50%)	50%	0.66

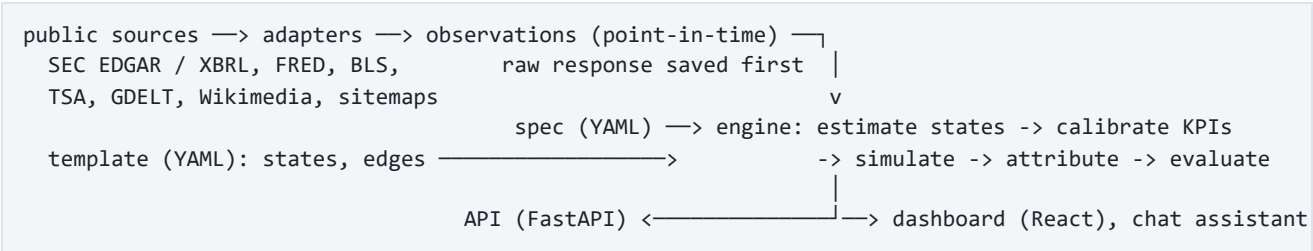
Reading it honestly. Link 1 holds. Link 2 holds against management guidance on accuracy, but the direction test is weaker. Chipotle beat its own guidance in 13 of 18 quarters, so the twin's 72% only matches "always predict a beat". Delta's 64% against a 50% base rate is suggestive but not conclusive. Against analyst consensus, the closest public stand-in for what the price expects, the twin shows no edge on the 6 quarters we could collect. Link 3 is untested. **We do not claim demonstrated alpha.** What we have built is the system that would generate it, and the instrument that measures honestly whether it does.

Where it is most likely to appear: where public sensors are more direct and more timely than the analyst process. Airlines are the case in point: TSA counts are daily, and the airline twins are the strongest in the portfolio.

What would demonstrate it:

1. Point-in-time consensus history for every twin, so link 2 can be scored against the market's expectation rather than management's.
2. An earnings-day event study: does the twin's probability of beating consensus predict the sign of the post-release return? This needs a price source whose terms allow it, logged in `provenance.md`.
3. Sealed live forecasts, which accumulate a record nobody can fit after the fact. That machinery is running: Delta's Q3 resolved on Oct 9, and Chipotle's Q3 resolves on Oct 28.

3. Architecture



Layer	Contents	What a new company or industry needs
`engine/`	estimation, calibration, simulation, attribution, evaluation, sealing	nothing (a test fails if a ticker, state name or series name appears here)
`adapters/`	SEC EDGAR and XBRL, FRED, BLS, TSA, Wikimedia, GDELT, a generic sitemap counter with Wayback history	usually nothing
`templates/`	an industry's states, default causal edges with priors, KPI structure	one YAML file per industry
`onboarding/`	ticker → SEC SIC code → industry pack → spec; optionally an LLM reads the 10-K and proposes additions	the ticker
`specs/`	per company: KPIs, signals, edge overrides, extraction rules	written by hand (2) or by the factory (15)

Two tests enforce the separation. `test_engine_no_company_code.py` searches the engine for tickers, template state names and series prefixes. `test_template_agnostic.py` runs estimation and simulation end to end on a dummy template with made-up state names.

4. Data and point-in-time correctness

- **Coverage:** 164,221 observations of 68 signal series from 6 public sources (FRED, BLS, TSA, GDELT, Wikimedia, public sitemaps). From SEC filings there are 2,564 reported KPI values for 17 companies (including extra KPIs from the LLM onboarding experiment), plus 56 management-guidance values and 6 consensus values.
- **Against the hackathon's dataset list:** the system draws on 10 of its entries: TSA checkpoint numbers, Wikimedia pageviews, GDELT 2.0, BLS (CES wages; JOLTS tested and not adopted), SEC EDGAR filings and XBRL, EDGAR full text (10-K for LLM onboarding, 8-K releases for KPIs and guidance), FRED, Census MRTS including food services (via FRED), Case-Shiller (via FRED) and Census housing starts (via FRED); jet fuel prices are EIA's, via FRED. Beyond the list: store and job-posting counts from Chipotle's public sitemaps, with history from the Internet Archive. Not used, with reasons in `provenance.md`: Google Trends (no official open API) and EIA or USDA AMS directly (planned; they need registered keys).
- **Raw first:** every fetch is saved untouched to `data/raw/<source>/<date>/` (362 snapshots) before parsing, so a parser can be fixed and rerun.
- **Point in time:** every observation stores `observed_at` (the period it describes) and `available_at` (when it could first be known). Release lags are set conservatively: 45 days after month end for monthly macro data, and the EDGAR acceptance time for filings. A backtest at time *t* uses only rows with `available_at ≤ t`. A test checks this.
- **Traceability:** every value carries a `source_ref`: the URL, the SEC accession number and exhibit, the raw snapshot path, and for text extractions the exact sentence it came from. The dashboard shows the sources behind each driver bar.
- **Legal data only:** robots.txt and terms were checked for every source, and nothing requires a login. Chipotle menu prices were not scraped, because they sit behind an API key in the site's JavaScript. Consensus estimates are hand-entered from public news quotes with the URL and quote kept.
- **Known caveat:** FRED and BLS serve the latest data vintage, so revised macro values can leak a little hindsight into old backtest quarters. Market prices and counts are not revised. A fix using ALFRED vintages is listed in `provenance.md`.

5. Model

Kept deliberately simple: about 30 scored quarters per twin cannot support a complex model.

1. **Signals → features:** each series becomes a decay-weighted level (half-life per signal), then a year-over-year change, standardized over the history available at forecast time.
2. **Features → states:** each state has a N(0, 1) prior. Each signal is a noisy reading of the state with precision p = c/(1 − c) · 0.5^(staleness/half-life), where *c* is the signal's confidence. The state mean is the precision-weighted average, so each signal's share of the state is exact and is stored.
3. **States → KPIs:** Bayesian linear regression with priors taken from the causal edges, in standardized units ("one sd of the parent moves the KPI by `prior_mean` of its sd"). Each KPI can be modeled as:
    - a **level**, with optional last-quarter and same-quarter-last-year terms;
    - a **change** since the last report;
    - an **anchored surprise** relative to the latest guidance or consensus published before the release.
4. **Identities and bounds:** accounting identities such as comps = transactions + average check are enforced on every simulated path by a minimum-variance projection. Physical bounds (a margin cannot exceed 100%) clip the paths, and the share clipped is reported.
5. **Simulation:** fully vectorized NumPy. States evolve as AR(1) processes with edge-propagated innovations, and scenario shocks decay with a half-life. A 10,000-path forecast takes about 0.03 s on a laptop, and a full refit about 0.06 s.
6. **Attribution:** a driver waterfall (each row linked to its sources) and leave-one-signal-out counterfactuals.

Shock half-lives for the scenario presets were measured from real events (`shocks/library.yaml`). For example, Chipotle's 2015 food-safety crisis decayed with a half-life of about 7.4 quarters, and COVID's hit to airlines and hotels with about 3.9.

6. Evaluation protocol

- **Walk-forward:** for each reported quarter, the forecast is made 1 hour before the earnings release, and everything is refit from scratch on data available at that moment. Tested from 2017Q1.
- **Excluded from scoring:** the COVID quarters (2020Q1–2021Q4), plus any regime break a spec declares (travel through 2022Q4). Specs can also leave those quarters out of calibration, and the travel specs do.
- **Baselines:** last quarter (`naive_last`), the same quarter last year (`seasonal_naive`), management guidance, and analyst consensus where public.
- **Metrics:** MAE, RMSE, bias, CRPS (scores the whole forecast distribution, not just the point) and 80% interval coverage. Skill = 1 − CRPS(twin)/CRPS(baseline) on the same quarters; positive means the twin is better.

7. Results

Full table: `docs/portfolio_scorecard.md` (17 twins, 37 KPI series; rescored 2026-10-18).

Result	Value
KPI series beating "last quarter" on CRPS	**25 of 37**
KPI series beating "same quarter last year" on CRPS	**35 of 37**
DAL revenue growth vs Delta's own guidance (14 guided quarters)	skill **+0.29** (twin better)
CMG comps vs Chipotle's guidance (20 quarters)	skill **+0.31** (twin better)
CMG comps vs analyst consensus (6 quarters)	skill **−0.37** (consensus better)

Selected KPIs:

Twin	KPI	Quarters	MAE twin	MAE last quarter	Skill vs last quarter	80% coverage
DAL	revenue growth	27	2.83	4.08	+0.31	0.67
DAL	operating margin	27	2.32	4.93	+0.37	1.00
AAL (factory)	operating margin	26	2.60	5.68	+0.49	0.85
PHM (factory)	revenue growth	27	4.74	4.45	+0.33	0.96
PHM (factory)	pre-tax margin	27	2.32	2.46	+0.68	0.96
CMG	comps	30	3.13	3.69	+0.21	0.80
CMG	revenue growth	30	3.86	3.92	−0.06	0.97

Reading the results honestly:

- Airlines are strongest, because TSA counts and jet fuel are timely, direct sensors.
- Restaurant and hotel revenue growth roughly ties "last quarter". Free public data carries little company-specific information there.
- Analyst consensus still beats the twin on Chipotle comps. Analysts see information that free public data does not.
- Interval coverage is uneven: some twins are too wide (coverage 1.00), and DAL revenue is a little narrow (0.67).
- **Selection bias:** the KPI structures for the two hand-built twins were chosen on the same backtest that is reported. The clean tests are (a) the 15 factory twins (14 rules-only, 1 LLM), which use industry defaults with no tuning, and (b) the sealed live forecasts below.

8. Live out-of-sample test: sealed forecasts

Before Delta reported Q3 2026, its forecast (revenue growth median 15.2%, 80% range 10.6 to 19.9, P(above guidance) 52%) was written to canonical JSON and its SHA-256 recorded. The hash was then timestamped through OpenTimestamps in Bitcoin block 969637 (Oct 2, 2026). Anyone can confirm that the forecast existed before the outcome, without trusting us.

`scripts/resolve_sealed.py`:

1. recomputes each hash,
2. accepts an outcome only if it became public after the seal,
3. scores the twin against guidance, last quarter and last year (`docs/sealed_results.md`).

Outcome (Delta reported Oct 9, 2026, 8-K accession 0000027904-26-000035). The hash verified.

KPI (GAAP)	Sealed median (80% range)	Actual	Inside 80%	Twin CRPS	Guidance	Last quarter	Last year
Operating margin	6.9 (−1.8 to 15.6)	**7.2**	yes	**1.60**	–	2.23	2.90
Revenue growth	15.2 (10.6 to 19.9)	**21.1**	no	3.93	6.07	**2.40**	14.72

- **Margin:** a hit. The twin was 0.3 points off and beat both naive baselines.
- **Revenue growth:** a miss. The actual was above the 80% range. The twin beat guidance but lost to "same as last quarter".
- **Cause:** GAAP revenue includes Delta's third-party refinery sales, which rose 76% with high fuel prices and added about 6.7 points of growth. The guidance the twin anchors on excludes the refinery; on that basis revenue grew 15.7%, 0.5 points from the twin's median. The twin anchored a GAAP KPI on non-GAAP guidance. The scored result stays as sealed. Future forecasts will match each anchor's definition to its KPI.

Chipotle's Q3 forecast was sealed on Oct 18, 2026 (comps median 2.94%, 80% range 0.64 to 5.15, P(above guidance of 2.65%) 57%; SHA-256 `fabe5e811fcf…`, OpenTimestamps), ten days before its Oct 28 report.

9. Validating extracted data: automation and manual review

`scripts/validate_ground_truth.py` runs automatic checks on **every** row (`docs/validation_report.md`):

Check	What it tests	Result (2,564 rows)
Text span	The source sentence appears verbatim in the saved filing, and its number (signed by its wording) equals the stored value	154 of 154 pass
Identity	comps = transactions + check on reported values, within the company's own rounding	14 of 14 pass
Bounds	Values inside physical limits	all pass
Outliers	Values more than 4 sd from the KPI's history, flagged for review	24 flagged: all real events (COVID, including Carnival's near-zero 2020 revenue; Starbucks' 2013 Kraft arbitration charge, Marriott's 2009 timeshare write-down, Pulte's 2010 post-Centex goodwill impairment and merger-inflated growth)

Building this caught three bugs **in the checker**, not in the data. Examples: "1.8% fewer transactions" was read as positive, and "December increased 14.7% and declined 4.8% for the full quarter" holds two numbers. That kind of variation in wording is why each rule is tested on real release text, and why every value keeps its source sentence.

Answer to "how much automation versus manual validation": automate every row, and send people a fixed-seed random sample of 20 (`docs/validation_sample.md`) plus anything flagged. Reviewers mark rows y or n, and `--apply` records them as verified. Reporting the agreement rate on the sample tells users how far to trust the automatic extraction.

10. LLM onboarding: the model proposes, code decides

`python scripts/make_twin.py <TICKER> --llm` adds one step to the factory. After the SEC lookup and the SIC-based pack choice, an LLM (`claude-sonnet-5-5`, prompt `prompts/onboard_spec_v1.md`) reads the company's latest 10-K (about 100–170k tokens) together with a menu of what the system can use: the template states, the industry packs, every series an adapter can fetch, and the XBRL concepts this company actually reports. It returns structured JSON, validated with pydantic, proposing:

- an industry pack, when no pack covers the company's SIC code;
- up to 4 extra driver series, each mapped to a state with a sign and strength;
- up to 2 extra KPIs built from XBRL concepts, with edges into them;
- edge strengths, and up to 4 operating metrics (suggestions only, not used by the model).

Every proposal must carry a quote copied from the 10-K. Code, not the model, then decides. A proposal is dropped if its quote is not in the filing, its series cannot be fetched, its XBRL concept has fewer than 16 quarters of history or is no longer reported, it names an unknown state or KPI, or a new KPI has no accepted edge. Accepted items are merged over the pack defaults. `specs/auto/<ticker>.onboarding.json` records every proposal, its quote, and the reason for each rejection; the raw prompt and response are saved under `data/raw/llm/` like any other source. A twin costs about $0.20–0.35 in API calls and builds in about 100 seconds.

Carnival (SIC 4400, no pack) is the first twin built this way. The model chose the hotels pack, quoting the 10-K's list of competitors ("hotels, resorts …"), and added jet fuel as an input cost and airfares as a drag on demand, each with a quote. Code rejected three proposals: an edge from capacity to revenue, because its quote ("1.0% capacity increase in ALBDs") was not in the filing, and two proposed KPIs that had no accepted edge. A rehearsal on Royal Caribbean took 103 seconds end to end.

Result (`docs/experiments.md` §2, `docs/llm_onboarding_scorecard.md`). We reran four existing rules-only twins (UAL, MAR, SBUX, TGT) with the LLM step and rescored both versions on the same data.

Twin	Revenue growth, rules → + LLM	Op margin, rules → + LLM
UAL	+0.45 → +0.45	+0.52 → +0.53
MAR	−0.07 → **−0.13**	+0.20 → +0.20
SBUX	+0.05 → +0.06	+0.55 → **+0.58**
TGT	+0.01 → +0.01	−0.05 → −0.05

(CRPS skill against "last quarter"; positive is better.) On the KPIs both versions share, the LLM changed skill by at most ±0.06, inside the noise for these sample sizes, and made MAR revenue slightly worse. **We do not claim an accuracy gain.** Its value is coverage: it onboards industries no pack covers, and it finds company-specific KPIs the twin forecasts better than naive (UAL fuel share of revenue +0.37, labor share +0.32; Target SG&A ratio +0.36, cost-of-sales ratio +0.38). It still needs review: on Starbucks it mixed two XBRL concepts of different scope for one KPI, which then forecast far worse than naive. New KPIs therefore stay out of the portfolio scorecard until a person has checked them.

11. Experiments, including negative results

`docs/experiments.md` records every signal tested. For example, JOLTS quits and job-openings rates were tested as hiring-difficulty proxies (the office-hours question on labor data) and did **not** improve the CMG or DAL

margins. DAL operating margin MAE went from 2.43 to 2.49–2.61. They were not adopted. Industry-wide hiring conditions are not company-specific, and they move with wages, which the model already uses. The LLM onboarding experiment (section 10) is recorded there too: adopted for coverage, not for accuracy.

12. Scope, roadmap and implementation plan

The system models **companies whose results come from observable operating drivers**: restaurants, retail, airlines, hotels, homebuilders and cruise lines today. It does not attempt the whole market:

- Banks need a new template (deposits, credit quality, margin).
- Binary-event companies such as clinical-stage biotech need a separate event engine.
- Both are on the roadmap in `docs/coverage_and_roadmap.md` and need no change to the existing engine.

Next steps, in order: turn the operating metrics the LLM suggests (comps, RevPAR, occupancy) into checked extraction rules; read guidance from earnings-call transcripts as well as press releases; add strong-fit industries that reuse existing adapters (trucking, rail); then the bank template and the event module.

Implementation plan for an investment team. Each phase ends with a decision based on scores, not on impressions.

Phase	Work	Needs from the team	Decision at the end
1. Test the edge (weeks 1–2)	Load point-in-time consensus history as a new expectation source (the `external_forecasts` table already holds guidance and consensus). Rerun the beat/miss test (section 2) against consensus for every twin. Add a price source and run the earnings-day event study: does the twin's probability of beating consensus predict the sign of the post-release return?	Consensus and price history under existing licences, cleared by compliance	Continue only if beat/miss calls clear the best constant rule against consensus
2. Deploy (weeks 2–4)	Containerize (database, API, dashboard); move storage to Postgres, since the migrations are already portable SQL; run the daily job on a server. Build twins for the fund's coverage names in supported industries; an analyst checks each spec, including every LLM-proposed KPI. Seal a forecast before every report.	A server; about one analyst-hour per new name	Sealed record after one reporting season
3. Daily use (months 2–3)	A pre-earnings sheet per name (probability of beating consensus, the drivers and their sources); scenario runs for PMs; an alert when a hidden state moves more than 1 sd. Add licensed earnings-call transcripts so guidance given on calls is captured.	Transcript licence; analyst feedback	Keep the KPIs and signals that the walk-forward score supports
4. Widen (month 3 on)	New templates on the same engine: banks, then binary events	—	Same scorecard for every new twin

Rules that carry over: every forecast is sealed before the outcome; a model change ships only with a full walk-forward rescore; every source is logged with its terms; licensed data never leaves the team's systems (public data and code stay separable).

13. Limitations

- **Short histories:** about 26–31 scored quarters per KPI. Skill differences under about 0.05 are noise.

- **A forecasting model, not a structural one:** scenario results are what the calibrated linear model implies. Moves far outside history (for example, fares up 30%, 2.5 sd) are extrapolation, and the dashboard and assistant say so.
- **Causal edges** between states come from priors, and are validated rather than discovered.
- **Seasonal margins** only tie the same-quarter-last-year baseline.
- **Rate limits:** the GDELT news-tone API rate-limits heavily, so some news-tone series can lag a few days. The daily job retries them.
- **Guidance from press releases only:** guidance given only on earnings calls is missed. Chipotle's Q3 2026 call guide of "around a plus 1%" comps is the live example: the twin compared itself with the 2.65% implied by the press release's full-year range.
- **No call transcripts, reviews or forums:** free transcript sources are licensed, and review and forum APIs restrict this use, so the text the system reads is filings and news tone only.
- **LLM hindsight:** the LLM reads today's 10-K, and the backtest scores the resulting structure on earlier quarters, the same caveat as choosing a pack. Its instructions forbid using known outcomes, but that cannot be verified.

14. Reproduce

```
pip install numpy "pydantic>=2" pyyaml requests fastapi uvicorn anthropic
python -m pytest -q                                  # 71 tests, no network (saved fixtures)
export CHUI_SEC_USER_AGENT="Your Name you@example.com"  # SEC requires a contact
export ANTHROPIC_API_KEY=...                         # only for --llm onboarding and the chat assis
python scripts/ingest.py specs/cmg.yaml --signals --ground-truth
python scripts/backtest.py specs/cmg.yaml --from 2017Q1 --exclude 2020Q1:2021Q4
python scripts/make_twin.py SBUX UAL MAR             # factory: ticker in, scored twin out
python scripts/make_twin.py RCL --llm                # same, with the LLM reading the 10-K
python scripts/validate_ground_truth.py
python scripts/reports/surprise_direction.py         # does the twin call beat or miss vs expectatio
python scripts/resolve_sealed.py
uvicorn api.serve:app --port 8000                    # dashboard and API
```